

Simulating 2D autonomously exploring mobile robot using DRL

Learning aim

To demonstrate and understand the use of DRL algorithms to solve a mobile robot exploration task in a custom simulated environment.

Learning objectives

In this practical, you should:

- Read the important part of the code involving the concepts introduced in our reinforcement learning lectures.
- Play around the hyperparameters of the DRL agent to try and get the mobile robot to explore the “default” (“playground”) environment aiming to navigate along the outer wall of the environment in a counterclockwise direction while avoiding collision with the walls and inner obstacles.

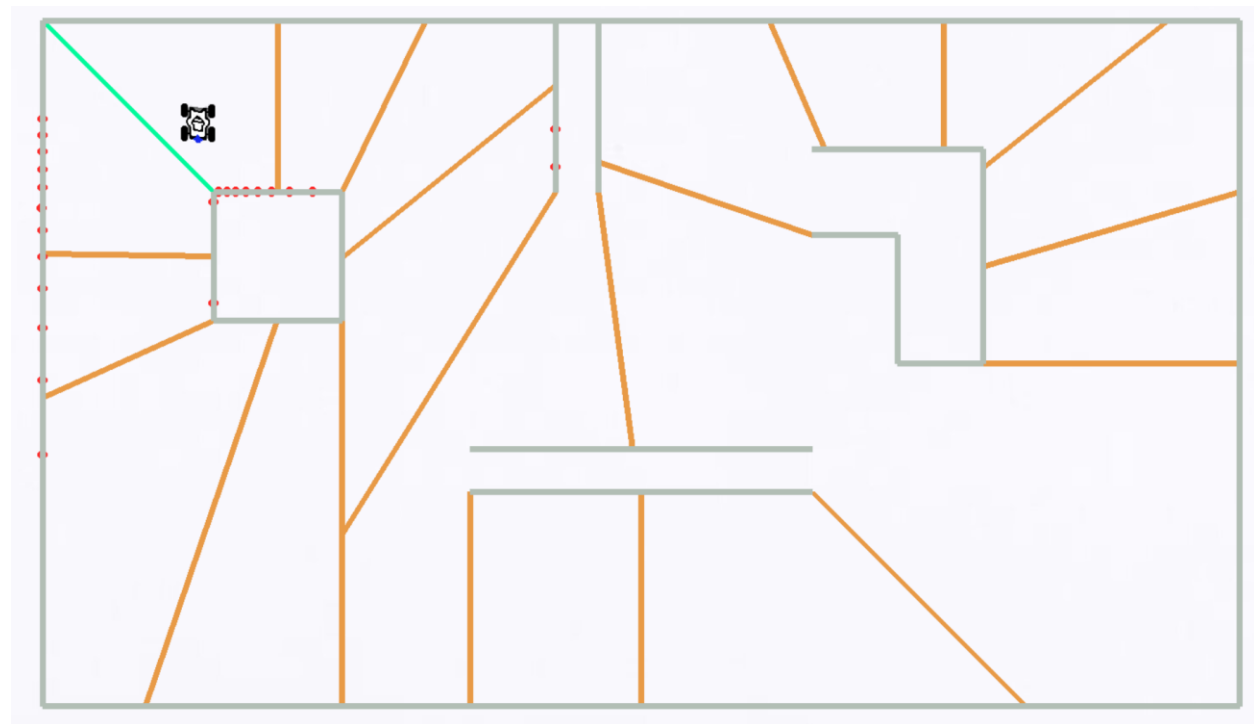


Figure 1. Layout of “playground” 2D exploration space

Practical assignment 1.4a

AI for Autonomous Robots

- You can play around the positions of the goal vectors of the mobile robot (line segments in orange), a form of reward shaping, and some of its default specs (acceleration and angular velocity constants, number of echo vectors) but keep the original layout of the walls and obstacles to have comparable results with other groups.

After this exercise you should to be able to:

- Grasp how RL concepts we discussed in our lectures are implemented in popular software libraries.
- Explain selected hyperparameters that you play around with to solve the toy exploration task defined in this practical assignment.

Type

Practice and Exercise

Expected time investment

8 to 10 hours

Required material and data

This practical assignment description and code provided by your supervisor.

Deliverables

Unlike past years, the assignment won't have a report but a short video/presentation (called video report in the following) recorded for each assignment. The group should deliver this output (combination of images, videos and slides) to explain what they implemented and how their solution performs in reality, adding footages of their (most successful) tests. The maximum length of each video (one video per assignment) is 3 minutes and no more than 20 MB. In total, the three assignments (described below) lead to a unique video (max duration 9 minutes) that must be delivered before the deadline.

The video should be technically informative: details on the algorithms used, workflows and other relevant elements must be clearly reported. In addition, the video report should evidence the capabilities of the developed solution, showcasing tests, also in "challenging" scenarios.

Software setup and description

Setup and installation

Download the zip file containing the code provided with this practical assignment description in canvas. This is an updated and slightly modified version of the code provided in:

https://github.com/BavanthaU/Exploring_agent_DRL. Install the following python packages following the instructions provided in the links given to each python package documentation.

- Ray[rllib] (<https://docs.ray.io/en/latest/ray-overview/installation.html>)
- PyTorch (<https://pytorch.org/get-started/locally/>) or Tensorflow (<https://www.tensorflow.org/install>) as Ray's backend

Practical assignment 1.4a

AI for Autonomous Robots

- Explore-Agent package provided in the provided code (Exploring_agent_DRL/Explore-Agent)

The last item is done by changing your working directory to the Explore-Agent directory and running:

```
pip install .
```

It is suggested to create a python virtual environment by either using some tools like miniconda (<https://docs.anaconda.com/free/miniconda/>) so that the packages above does not conflict with your default/local python installation.

The following installation commands are tested in a Windows machine using miniconda and PyTorch as backend for Ray[rllib].

```
conda create --name=my_env python=3.9
activate my_env
cd Explore-Agent
pip install .
pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
```

Designed environment

The custom environment is designed by creating a subclass of OpenAI's library called the gym. To use the gym, one must create an instance of an environment and interact with it using the base methods such as reset to initialize the environment, step to take action in the background and receive the reward and new state, and render to visualize the setting. These methods define the behavior of the environment and allow for interaction with the reinforcement agent. Adopting the gym structure ensures compatibility with existing reinforcement learning algorithms for the initial testing and makes it easier to switch between different configurations. So the continuous environment is designed to test the state-of-the-art DRL agents considering the below factors.

Action Space

The objective is to control a mobile robot in a continuous space through its forward/backward propulsion (action 1) and left/right rotation (action 2). To achieve this, the actions are represented as a two-dimensional linear continuous vector, reflecting the continuous nature of the control inputs. Where, $a = [a_1, a_2]$; $a_1 > 0$ represents forward propulsion and $a_1 < 0$ represents backward propulsion; $a_2 > 0$ represents right rotation and $a_2 < 0$ represents left rotation.

Observation Space

The observation is the information from the environment illustrated in Figure 1, that the agent or the neural network can utilize to determine its next action. The agent's decision is solely based on the current observation since the agent does not have a memory but experience, so it can be represented as a deterministic function that calculates the action $[a]$ based on the observation as in equation 3.

$$a = f(O); \text{ where } O = [l_1, l_2, l_3, l_4, l_5, l_6, l_7, v, \alpha, \beta] \quad 3$$

Practical assignment 1.4a

AI for Autonomous Robots

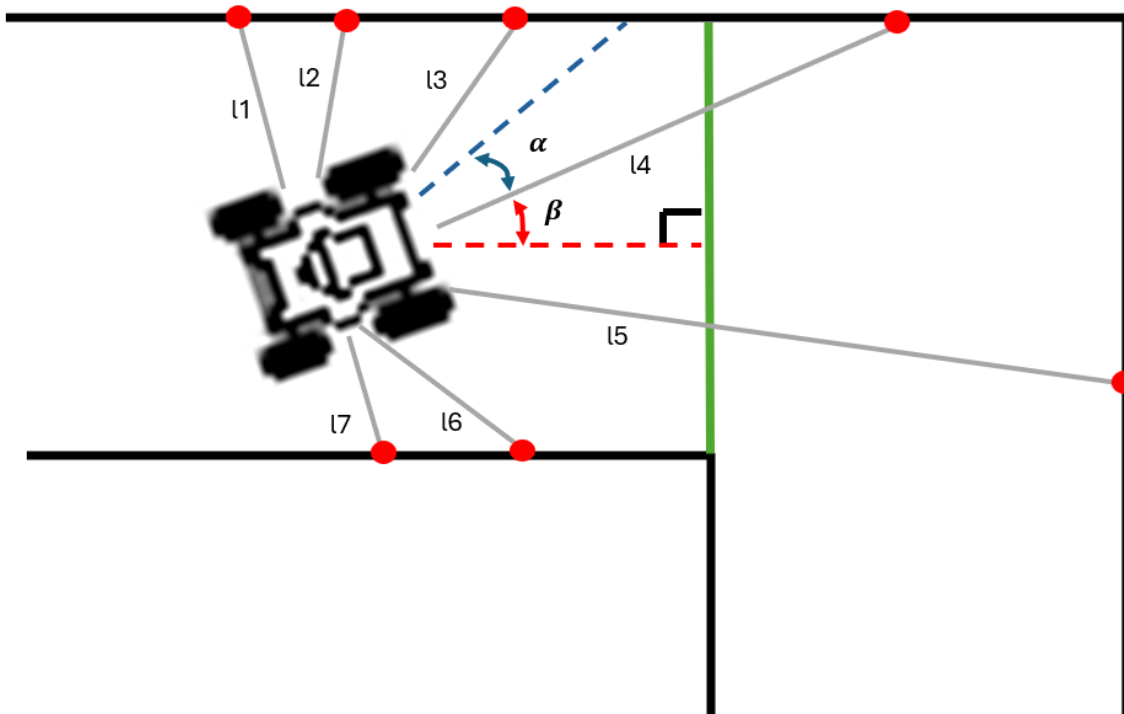


Figure 1. Observation space values illustrated

In the given context of controlling the mobile robot in this example, the observation space O is defined as follows:

- Clearance between the mobile robot and the environment in various angles, represented as $l_1, l_2, l_3, l_4, l_5, l_6, l_7$
 - $l_1 = (\theta = -90^\circ, d1), l_2 = (\theta = -60^\circ, d2), \dots, l_7 = (\theta = 90^\circ, d7)$
- The speed magnitude of the mobile robot is represented as v .
 - $v = ||v||$
- The relative angle between the mobile robot's orientation and its velocity is represented as α .
 - $\alpha = \tan^{-1}\left(\frac{v_y}{v_x}\right)$
- The angle between the mobile robot's orientation and the direction of the next goal is represented as β .
 - $\beta = \tan^{-1}\left(\frac{y_{goal} - y_{drone}}{x_{goal} - x_{drone}}\right)$

Note: In the above representation, v_x and v_y represent the velocity of the mobile robot in the x and y directions ; y_{drone}, y_{goal} and x_{drone}, x_{goal} represent the y and x coordinates of the goal and the mobile robot, respectively. Also, length ($l_1 - l_7$) values are normalised within $[-1, 1]$ to ensure the agent will not be saturated while training.

Reward Function

Defining a reward function that accurately captures the desired behaviour of the agent can be challenging. A common issue arises when maximizing the reward function leads to outcomes that

Practical assignment 1.4a

AI for Autonomous Robots

are not aligned with the intended behaviour of the agent. Given these considerations, the reward function for the mobile robot in this scenario has been defined by setting up checkpoints along the track that represents new regions (room or corridor) the agent must pass to receive rewards. Instead of giving rewards based on velocity, which could lead to problematic behavior such as crashing into walls, the agent is rewarded for exploring new regions by passing through checkpoints.

$reward_{total} += \max(1, (step_limit - steps)) / step_limit$; where in the example $step_limit$ set to be 500 and $steps$ to be the current iteration of the action

Training a PPO-based agent

To train a PPO-based agent using default hyperparameters run the following script:

```
python Explore_PPO_agent_training.py
```

Testing a PPO-based agent

To test a trained PPO-based agent using the best saved model run the following script:

```
python explore_agent_rollout.py  
  
make sure to update :  
chkpt_file = "tmp/ppo/checkpoint_best"
```

Assessed Tasks

Attempt to complete the 2D exploration task by training a DRL agent selecting an algorithm of your choice. The sample in the instructions above is using PPO but you can select any off-the-shelf algorithm readily available in Ray. The task is considered completed by finishing one full lap traversing the map layout shown in Figure 1. Play around selected key hyperparameters of your chosen DRL algorithm and describe how varying a hyperparameter affect the results (reward metrics in the training log and visualization of testing a trained DRL agent). You can also play around the positions of the goal vectors of the mobile robot (line segments in orange), a form of reward shaping, and some of it's default specs (acceleration and angular velocity constants, number of echo vectors) but keep the original layout of the walls and obstacles to have comparable results with other groups.